CSE 816 Mini-Project: CI/CD Pipeline for a Calculator REST API

Name: ShivekRanjan **Roll No.:**

GitHub: https://github.com/ShivekRanjan/calculator-devops

Docker Hub: https://hub.docker.com/r/shivek414/calculator

1. Overview

The application is a Spring Boot 3.5 REST API (Java 21, Maven) with `/api/v1/add`, `/subtract`, `/multiply`, `/divide` and `/modulo`, each taking query parameters `a` and `b`, plus Actuator's `/actuator/health` and `/actuator/info`. Responses are JSON, e.g. `{"operation":"add","a":2,"b":3,"result":5}`. Maths is done with `BigDecimal`, so `0.1 + 0.2` returns `0.3`. Division or modulo by zero, a missing or empty parameter, a non-numeric value, and NaN/Infinity all return HTTP 400 with one consistent error body (`timestamp, status, error, message, path`) produced by a `@RestControllerAdvice`. There are 40 tests: 29 JUnit 5 unit tests for the service (every operation, zero divisors, non-finite inputs) and 11 MockMvc integration tests that call the real endpoints, including the error cases and the health endpoint. `mvn test` writes JUnit XML to `target/surefire-reports` for Jenkins.

Around the code is a pipeline in which a push to GitHub is built, tested, packaged into an image, pushed to Docker Hub and deployed with Docker Compose, with no manual step. Everything runs on one macOS machine with Docker Desktop.

2. Architecture

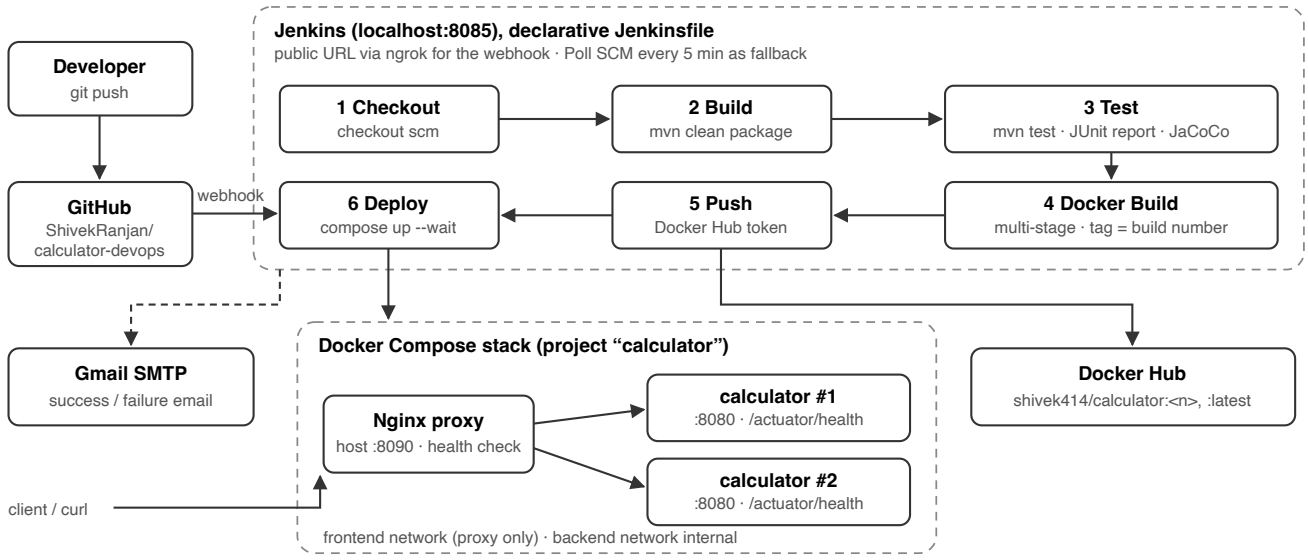


Fig. 1. Pipeline and deployment architecture.

Jenkins runs on the host and talks to the same Docker engine that runs the stack. GitHub cannot reach `localhost`, so an ngrok tunnel gives Jenkins a public URL for the webhook; Poll SCM (every 5 minutes) is the fallback if the tunnel is down. Only the Nginx proxy publishes a port. The two API replicas sit on an `internal` backend network and cannot be reached from outside.

3. Pipeline stages (Jenkinsfile)

Stage	What it does
Trigger	`githubPush()` (webhook) and `pollSCM('H/5 * * * *')`. `disableConcurrentBuilds()` stops two deploys from overlapping.
Checkout	`checkout scm` fetches the commit that triggered the build.
Build	`mvn -B clean package -DskipTests` produces `target/calculator.jar` using the Jenkins tools `JDK-21` and `Maven-3.9`.
Test	`mvn -B test`. `junit` publishes the XML reports and the JaCoCo report is archived. A failing test makes Maven exit non-zero, so the build fails.
Docker Build	`docker build` tagged `shivek414/calculator:$BUILD_NUMBER` and `:latest`. The build number is passed in as a build-arg and shown at `/actuator/info`.
Push	`withCredentials` binds the `dockerhub-credentials` token from the Jenkins credentials store, then `docker login --password-stdin`, push both tags, `docker logout`. The secret is masked in the log and never stored in the repository.
Deploy	`docker compose up -d --remove-orphans --wait` blocks until every container reports healthy, then a smoke test through Nginx calls `/actuator/health` and `/api/v1/add`.
Post	`emailext` sends an HTML success or failure email (the failure email attaches the log) through Gmail SMTP.

Idempotency. The Compose project name is fixed (`calculator`), so a re-run updates the same stack instead of creating a second one. Compose only recreates containers whose image changed, and `--wait` means a deploy never finishes with an unhealthy container.

4. Containerization and deployment

- **Dockerfile (multi-stage):** a `maven:3.9-eclipse-temurin-21` stage copies `pom.xml` first and runs `dependency:go-offline`, so the dependency layer is cached until the POM changes, then builds the JAR. The runtime stage is `eclipse-temurin:21-jre-alpine` with only `app.jar`. It runs as a non-root `spring` user, has an explicit `EXPOSE 8080` and a `HEALTHCHECK`, and `.dockerignore` keeps the build context to `pom.xml` and `src/`. `docker run -p 8082:8080 shivek414/calculator` runs it on its own.
- **docker-compose.yml:** the `calculator` service runs 2 replicas with a health check on `/actuator/health`, `restart: unless-stopped`, environment variables (`SPRING_PROFILES_ACTIVE=prod`, JVM memory options), a 512 MB memory limit, and an Nginx `proxy` service that waits for `service_healthy`. The image and tag come from `DOCKER_IMAGE` / `IMAGE_TAG`, which Jenkins sets, and the host port from `PROXY_PORT`.
- **Nginx:** round-robins over the replicas through Docker's DNS (`resolver 127.0.0.11 valid=5s`), retries the other replica on a 502/503, and adds an `X-Served-By` header that shows the load balancing.

5. Version control

The history was built up in steps: project skeleton, service and unit tests, REST layer and integration tests, Dockerfile, Jenkinsfile, documentation. The Compose stack was developed on `feature/compose-deploy` and merged into `main` through pull request #1. `.gitignore` excludes `target/`, IDE files, `.env` and key files. There are no credentials in the repository: the Docker Hub token and the Gmail app password live only in the Jenkins credentials store, and the notification address is a Jenkins global environment variable.

6. Challenges faced

1. **Port conflicts.** Port 8080 was already used by Jenkins and another container, and 8081 by an unrelated stack, so the first `compose up` failed with “port is already allocated”. The proxy port became a variable (`PROXY_PORT`, default 8081), set to 8090 in Jenkins' global environment, and Jenkins itself runs on 8085.

2. **Wrong JDK.** The machine's default `java` is 17 but the project targets 21. The Jenkins tool `JDK-21` points at the Homebrew OpenJDK 21 installation, so the pipeline doesn't depend on whatever is on the `PATH`.
3. **Docker login shared with the host user.** Jenkins runs as the same macOS user, so `docker login` / `docker logout` in the Push stage would overwrite, and then delete, the user's own Docker Hub login. The Push stage now sets `DOCKER_CONFIG` to a directory inside the job workspace, which keeps the pipeline's credentials completely separate.
4. **Minimal PATH for a background service.** A Jenkins started outside a login shell does not see `/usr/local/bin`, where Docker Desktop installs the `docker` CLI. The start script sets `PATH` explicitly.
5. **Webhook to a local Jenkins.** GitHub cannot reach `localhost`. An ngrok tunnel forwards `/github-webhook/` to Jenkins. Because the free URL changes on restart, Poll SCM stays as a fallback.
6. **Nginx and changing container IPs.** A plain `upstream` resolves names once at startup, so after a redeploy Nginx would keep sending traffic to dead IPs. Using a variable in `proxy_pass` with Docker's resolver makes it re-resolve every 5 s. The config is baked into a small Nginx image instead of bind-mounted, so Compose works whatever directory it is started from.

7. Results

Builds #1 to #3 passed every stage. Build #1 took 2 min 16 s (cold Maven and Docker caches), build #2 took 59 s, and build #3 took 1 min 22 s. Build #3 was the demo: a new `/modulo` endpoint with its test was pushed to `main`, the webhook started the build 10 s later, and about 1.5 minutes after the push `/actuator/info` reported `"build":"3"` and `/api/v1/modulo?a=10&b=3` returned `1`, with nothing done by hand. The images are public on Docker Hub, the stack runs healthy, and a status email arrives after every build.

8. Screenshots

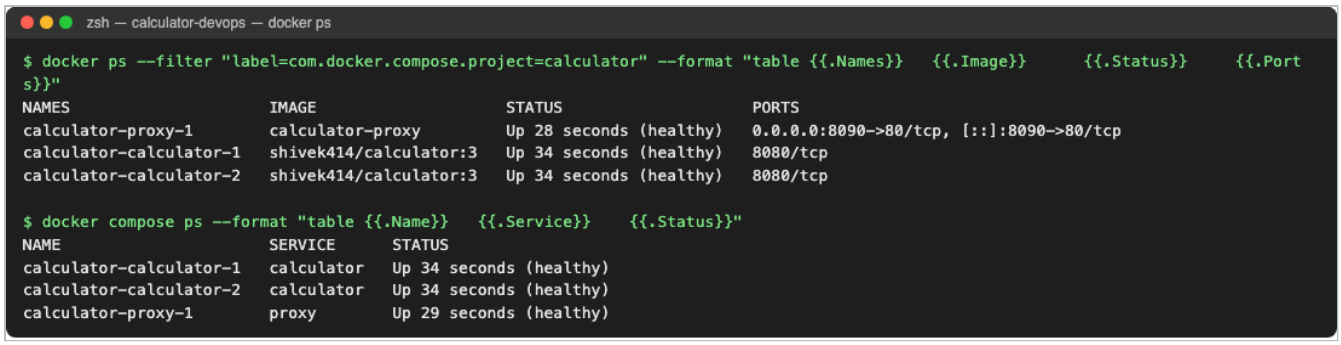


Fig. 2. `docker ps` / `docker compose ps`: two calculator replicas running build 3, and the Nginx proxy, all healthy.

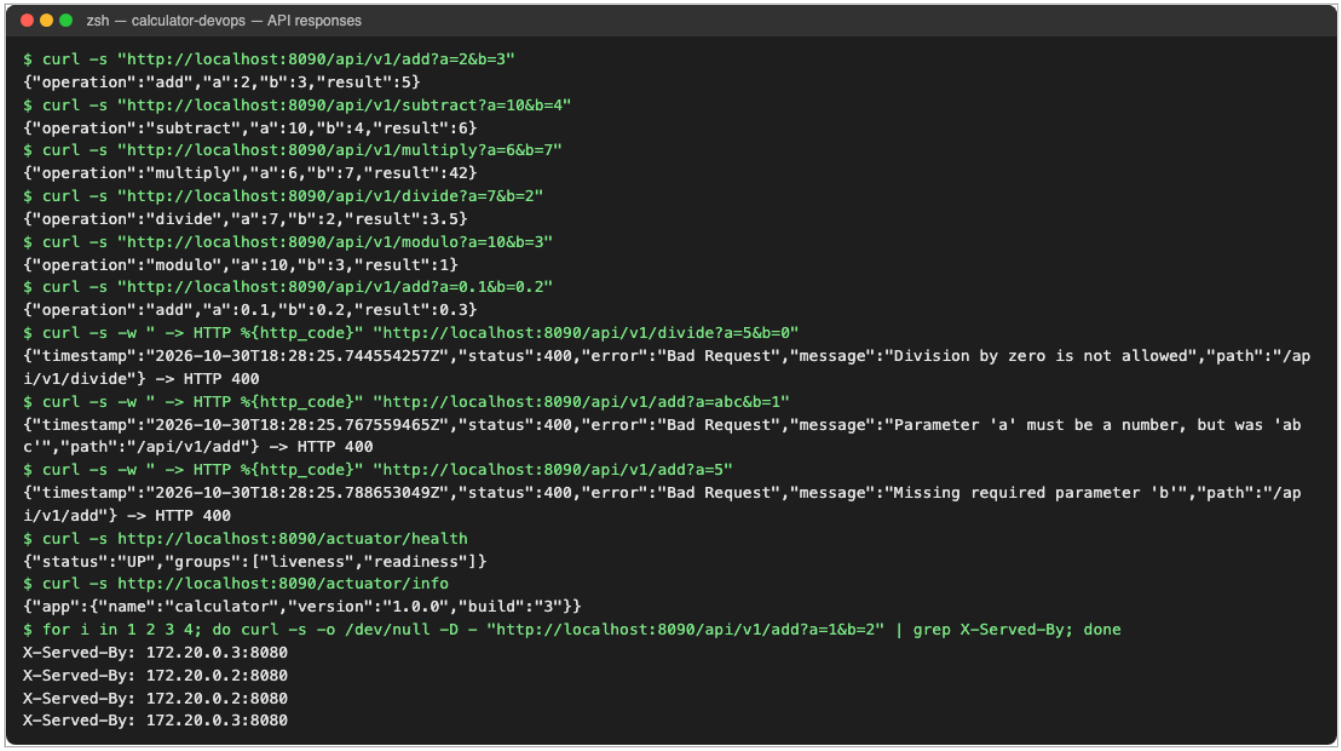


Fig. 3. API calls through Nginx on port 8090: results, HTTP 400 errors, health, the build number in `/actuator/info`, and `X-Served-By` alternating between replicas.

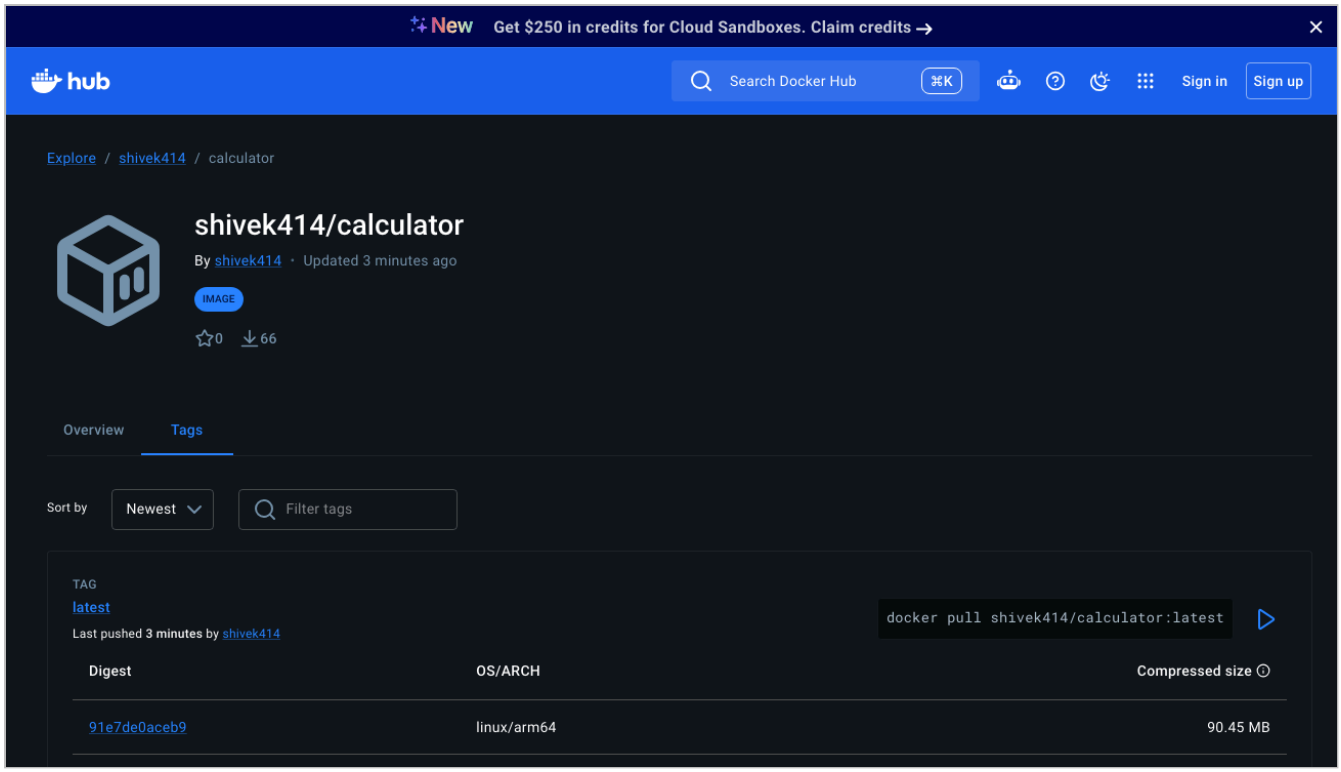


Fig. 4. Public Docker Hub repository `shivek414/calculator`, pushed by the Jenkins Push stage.



Fig. 5. Commit history: step-by-step commits and the merged feature branch.

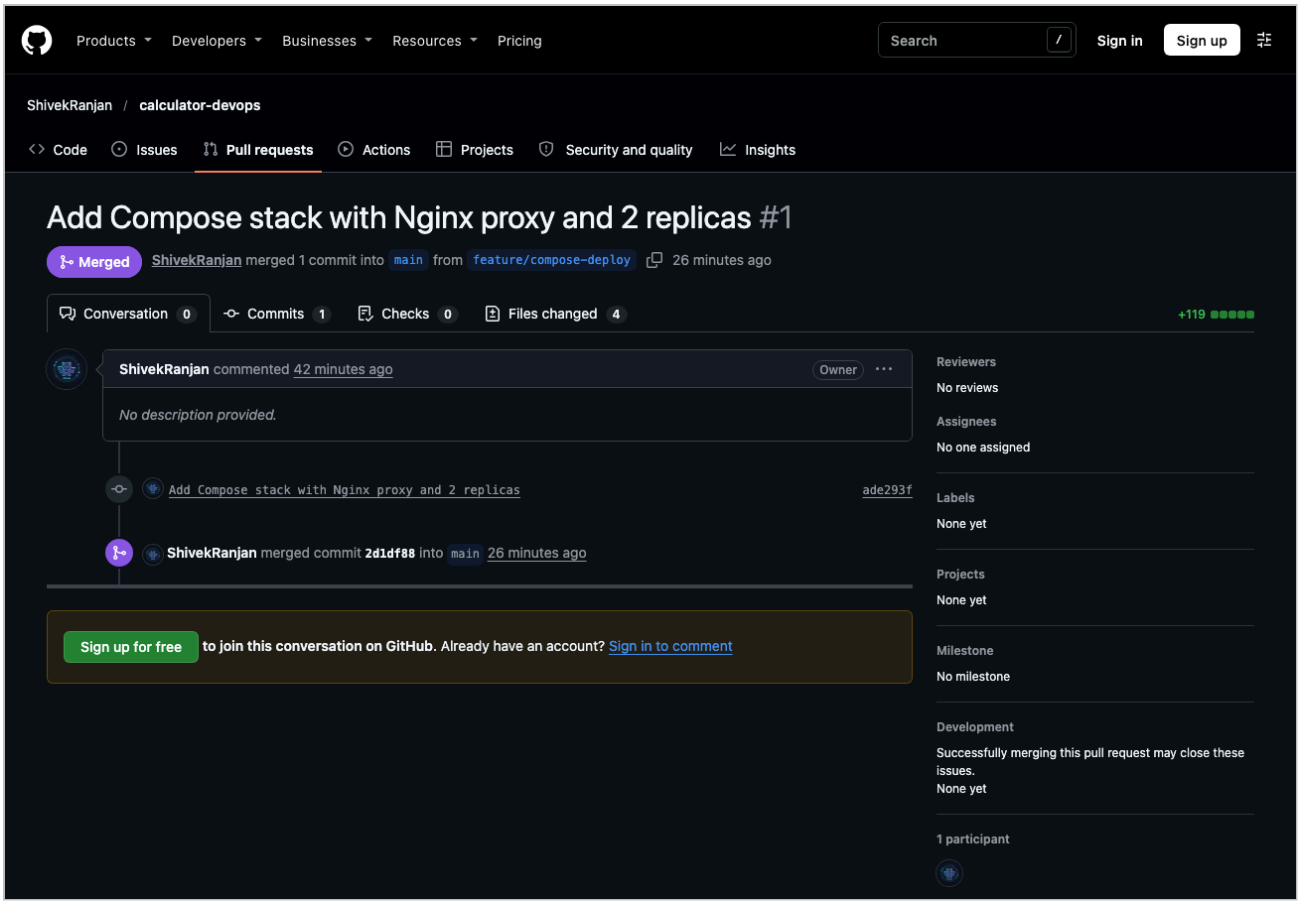


Fig. 6. Pull request #1 from `feature/compose-deploy`, merged into `main`.